

리눅스 4 - 인터럽트

남궁명수

[용어 정리]

구분	발생 원인	발생 시점	예
하드웨어 IRQ	CPU 외부 장치	비동기적	NIC, NVMe, 키보드
CPU 예외	현재 실행한 명령어	동기적	Page Fault, 0으로 나누기
시스템 콜	프로그램의 의도적인 요청	동기적	read(), write()
Linux SoftIRQ	커널의 지연 처리	커널이 등록	NET_RX, TIMER, RCU

특히 CPU 예외, 시스템 콜과 Linux SoftIRQ는 완전히 다른 개념이다.

[1] 하드웨어 인터럽트란?

하드웨어 인터럽트는 CPU 외부의 장치가 처리가 필요한 사건을 CPU에 알리는 비동기적 신호다.



현대 장치는 전용 핀뿐만 아니라 MSI/MSI-X처럼 메시지를 전송하는 방식으로도 IRQ를 발생시킨다. Linux에서 IRQ 번호는 이러한 하드웨어 인터럽트 소스를 나타내기 위한 커널 식별자다.

대표적인 하드웨어 사건은 다음과 같다.

- 네트워크 패킷 도착
- 디스크 I/O 완료
- 키보드·마우스 입력
- USB 장치 상태 변경
- 타이머 만료
- 장치 오류

[2] 비동기적이라는 의미

하드웨어 IRQ는 현재 실행 중인 명령어가 원인이 되어 발생하는 것이 아니다.

CPU

└─ 사용자 프로그램의 덧셈 명령 실행 중

네트워크 카드

└─ 외부에서 패킷 도착

|

└─ IRQ 발생

일반적인 마스커블 인터럽트는 CPU가 인터럽트를 받을 수 있는 상태일 때 명령어 경계에서 인식한다.

사용자 프로그램 실행

|

| 외부 장치 IRQ 도착

▼

현재 명령어의 실행 경계 도달

|

▼

CPU가 최소한의 실행 상태 저장

|

▼

커널 진입 코드가 추가 레지스터 저장

|

▼

Hard IRQ Handler 실행

|

▼

실행 상태 복원

|

▼

중단된 실행 흐름으로 복귀

다만 IRQ 처리 후 스케줄링이 필요하다면 다른 스레드가 먼저 실행될 수도 있다.
따라서 항상 즉시 같은 사용자 프로그램으로 돌아간다고 단정하면 안 된다.

[3] 인터럽트 컨텍스트

Hard IRQ Handler는 인터럽트 컨텍스트에서 실행된다.

프로세스 컨텍스트

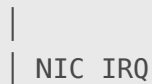
- 특정 스레드의 실행 흐름
- 필요하면 잠들고 스케줄링될 수 있음

인터럽트 컨텍스트

- 외부 장치 사건을 처리하기 위한 실행 흐름
- 일반적인 의미로 잠들 수 없음

IRQ가 사용자 프로세스 A를 실행하던 CPU에서 발생했다고 해보자.

프로세스 A 실행 중



Hard IRQ Handler 실행

현재 CPU에서 프로세스 A가 중단됐지만, IRQ Handler가 프로세스가 A를 위해 실행되는 것은 아니다.

프로세스 A가 디스크를 기다린다

- A를 대기 상태로 만들 수 있음

Hard IRQ Handler가 디스크를 기다린다

- 잠들 대상인 정상적인 태스크 실행 흐름이 아님
- 허용되지 않음

Hard IRQ Handler에서는 다음과 같이 잠들 수 있는 작업을 해서는 안 된다

- 일반 뮉텍스 획득 대기
- 디스크 I/O 완료 대기
- 사용자 공간 응답 대기
- schedule()을 통한 문맥 교환
- 잠들 가능성이 있는 메모리 할당
- 오래 걸리는 동기 작업

잠들어야 하는 작업은 Thread IRQ나 일반 Workqueue처럼 스레드가 실행하는 방식으로 넘겨야 한다.

[4] Hard IRQ Handler: 즉시 처리 부분

Hard IRQ Handler는 인터럽트가 발생한 직후 실행되는 가장 긴급한 처리 부분이다. 흔히 Top Half라고도 부른다.

주요 역할은 장치에 따라 다르지만 일반적으로 다음과 같다.

- 어떤 장치·큐가 IRQ를 발생시켰는지 확인
- 인터럽트 원인과 장치 상태 확인
- 인터럽트 상태 승인 또는 정리
- 필요한 경우 해당 IRQ 소스를 일시적으로 막음
- 사라질 수 있는 최소한의 상태 보관
- 후속 작업 등록
- 빠르게 종료

장치

| "처리할 사건이 발생했습니다."

▼

Hard IRQ Handler

|— 인터럽트 원인 확인

|— 장치 상태 확인·승인

|— 후속 처리 등록

|— 빠르게 종료

Hard IRQ Handler가 오래 실행되면 그 CPU에서 다음 작업이 지연될 수 있다.

- 일반 사용자 스레드
- 커널 스레드
- 같은 우선순위 또는 더 낮은 우선순위의 IRQ
- 타이머와 네트워크 처리

따라서 Hard IRQ에서는 다음 인터럽트를 정상적으로 받기 위해 지금 해야 하는 최소 작업만 처리한다.

[5] 후속 처리 방식

Hard IRQ에서 미룬 작업은 성격에 맞는 방법으로 처리한다.

방식	실행 환경	잠들기	대표 용도
SoftIRQ	소프트IRQ 컨텍스트	불가능	네트워크, 타이머, RCU
ksoftirqd	CPU별 커널 스레드	구조상 스레드지만 SoftIRQ 제약을 따름	밀린 SoftIRQ 실행
Threaded IRQ	IRQ 전용 커널 스레드	가능	장치 드라이버 후속 처리
일반 Workqueue	kworker 커널 스레드	가능	일반 비동기 커널 작업
Tasklet	SoftIRQ 기반	불가능	기존 드라이버의 지연 처리

[소프트IRQ 컨텍스트]

하드웨어 인터럽트 처리 중 지연된 작업(바텀 하프)을 리눅스 커널이 `_do_softirq()` 함수를 통해 비동기적으로 실행하는 특별한 실행 상태이다. 이 상태에서는 일반적인 프로세스 전환(Context Switching)이나 블로킹(Blocking) 동작이 허용되지 않는다.

[커널 스레드]

운영체제의 커널(kernel)이 직접 생성하고 관리하는 실행 단위이다.

유저 공간의 메모리나 UI와는 상호작용하지 않으며, 주로 시스템 메모리 관리, 입출력 처리, 백그라운드 작업 등 운영체제 핵심 기능의 안정적인 실행을 담당한다.

중요한 점은 다음과 같다.

SoftIRQ가 “중요하지 않은 작업”이라는 뜻은 아니다.

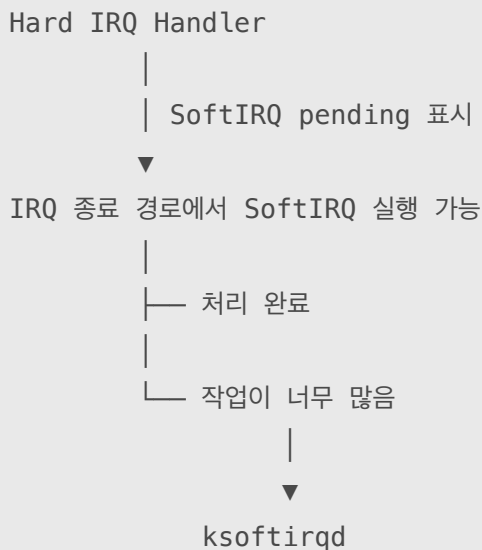
긴급하지만 Hard IRQ에서 전부 실행하기에는 시간이 오래 걸리는 작업을 처리한다.

일반 Workqueue는 워크 스레드에서 실행되므로 잠들 수 있다.

반면 SoftIRQ 컨텍스트에서 실행되는 BH Workqueue 등은 잠들 수 없다.

[6] SoftIRQ

SoftIRQ는 Linux 커널이 빠른 후속 처리를 수행하기 위해 사용하는 정적 지연 처리 메커니즘이다.



대표적인 종류는 다음과 같다.

NET_RX	네트워크 패킷 수신
NET_TX	네트워크 송신 완료 등
TIMER	일반 타이머
HRTIMER	고해상도 타이머 관련 처리
BLOCK	블록 장치 관련 후속 처리
IRQ_POLL	IRQ 폴링
SCHED	스케줄러 관련 처리
RCU	RCU 콜백 처리
TASKLET	기존 Tasklet 처리

현재 시스템에서 CPU 별 SoftIRQ 처리 횟수를 확인하려면:

```
cat /proc/softirqs
```

	CPU0	CPU1	CPU2	CPU3
NET_RX:	150000	210000	98000	170000
TIMER:	300000	290000	310000	280000
RCU:	120000	115000	122000	119000

/proc/softirqs는 부팅 이후 CPU별 SoftIRQ 처리 횟수를 보여준다.

[7] ksoftirqd

각 논리 CPU에는 일반적으로 SoftIRQ 처리를 돕는 ksoftirqd/N 커널 스레드가 있다.

```
CPU 0 → ksoftirqd/0
CPU 1 → ksoftirqd/1
CPU 2 → ksoftirqd/2
CPU 3 → ksoftirqd/3
```

SoftIRQ가 적으면 IRQ 종료 경로나 커널 실행 흐름에서 빠르게 처리할 수 있다.

하지만 SoftIRQ를 무제한 처리하면 사용자 프로그램이 실행되지 못할 수 있다.

```
NET_RX SoftIRQ 실행
|
├─ 계속 패킷 도착
├─ 계속 패킷 처리
└─ 사용자 스레드 실행 지연
```

커널은 한 번에 처리할 시간, 반복, 작업량을 제한하고, 남은 작업을 ksoftirqd가 이어서 처리하도록 할 수 있다.

SoftIRQ 작업 과다



확인:

```
ps -eLo pid,tid,psr,stat,comm | grep ksoftirqd
```

ksoftirqd의 CPU 사용률이 높다면 네트워크 트래픽, IRQ 분산, 드라이버 또는 특정 SoftIRQ 부하를 함께 확인해야 한다.

[8] Threaded IRQ

Threaded IRQ는 인터럽트 처리의 상당 부분을 IRQ 전용 커널 스레드에서 실행하는 방식이다.

장치 IRQ



드라이버는 request_threaded_irq() 등을 이용해 짧은 1차 Handler와 스레드 Handler를 등록할 수 있다.

Threaded IRQ는 다음 작업에 적합하다.

- 처리 시간이 비교적 긴 장치 제어
- 잠들 수 있는 락이 필요한 작업
- 버스 통신 완료를 기다리는 작업
- 복잡한 드라이버 상태 처리
- 실시간 우선순위 조절이 필요한 IRQ 후속 처리

[9] Workqueue와 kworker

Workqueue는 일반적인 비동기 커널 작업을 커널 워커 스레드에서 처리하기 위한 방식이다.

Hard IRQ 또는 커널 코드

|
| 작업 등록



Workqueue

|



kworker 커널 스레드

|

- ├─ 잠들 수 있음
- ├─ 뮤텍스 사용 가능
- └─ 시간이 걸리는 작업 가능

대표적인 용도는 다음과 같다.

- 장치 재설정
- 펌웨어 관련 후속 처리
- 전원 관리 작업
- 시간이 걸리는 드라이버 상태 변경
- 일반적인 비동기 정리 작업

```
ps -eLo pid,tid,psr,stat,wchan:25,comm | grep kworker
```

단, 모든 kworker가 인터럽트 후속 처리를 하는 것은 아니다.

Workqueue는 인터럽트뿐 아니라 다양한 커널 작업에 공통으로 사용된다.

커널 워커 스레드

운영체제 커널 내부에서 백그라운드로 실행되며, 워크큐(Workqueue)에 등록된 비동기 작업들을 가져와 처리하는 전용 실행 단위(스레드)이다.

[전체 구조]

flowchart TD

A["하드웨어 인터럽트 발생"] --> B["Hard IRQ Handler
최소한의 긴급 작업"]

B --> C{"남은 작업 처리 방법"}

C --> D["SoftIRQ 요청"]

C --> E["Threaded IRQ 깨우기"]

C --> F["Workqueue에 작업 등록"]

C --> G["추가 작업 없음"]

D --> H{"즉시 처리 가능한가?"}

H --> |가능| I["IRQ 종료 시 SoftIRQ 실행"]

H --> |작업 과다·시간 초과| J["ksoftirqd가 나중에 실행"]

E --> K["IRQ 전용 커널 스레드 실행"]

F --> L["kworker가 작업 실행"]

[Hard IRQ Handler]

- Hard IRQ Handler
- Primary Handler
- Top Half
- 상반부 처리

대표적인 작업

- 어떤 장치가 인터럽트를 발생시켰는지 확인
- 장치의 상태 레지스터 확인
- 인터럽트 발생 사실을 장치에 확인해 주기
- 필요하면 장치 인터럽트를 잠시 마스킹
- 수신된 데이터의 위치나 상태 저장
- SoftIRQ 발생 요청
- Threaded IRQ 깨우기
- Workqueue에 작업 등록

다음 단계 선택

네트워크 패킷 처리	→ SoftIRQ 발생
장치별 복잡한 처리	→ Threaded IRQ 깨우기
시간이 오래 걸리는 작업	→ Workqueue에 등록
처리가 이미 끝남	→ 바로 반환

[10] 하드웨어 IRQ와 CPU 예외의 차이

[하드웨어 IRQ]

현재 실행 중인 명령어와 직접 관계없는 외부 사건이다.

CPU: 덧셈 명령 실행

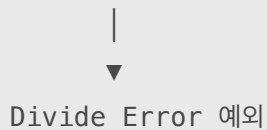
NIC: 패킷 도착



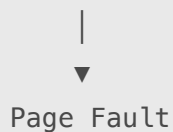
[CPU 예외]

현재 실행 중인 명령어 때문에 발생한다.

CPU가 0으로 나누기 명령 실행



CPU가 매핑되지 않은 가상주소 접근



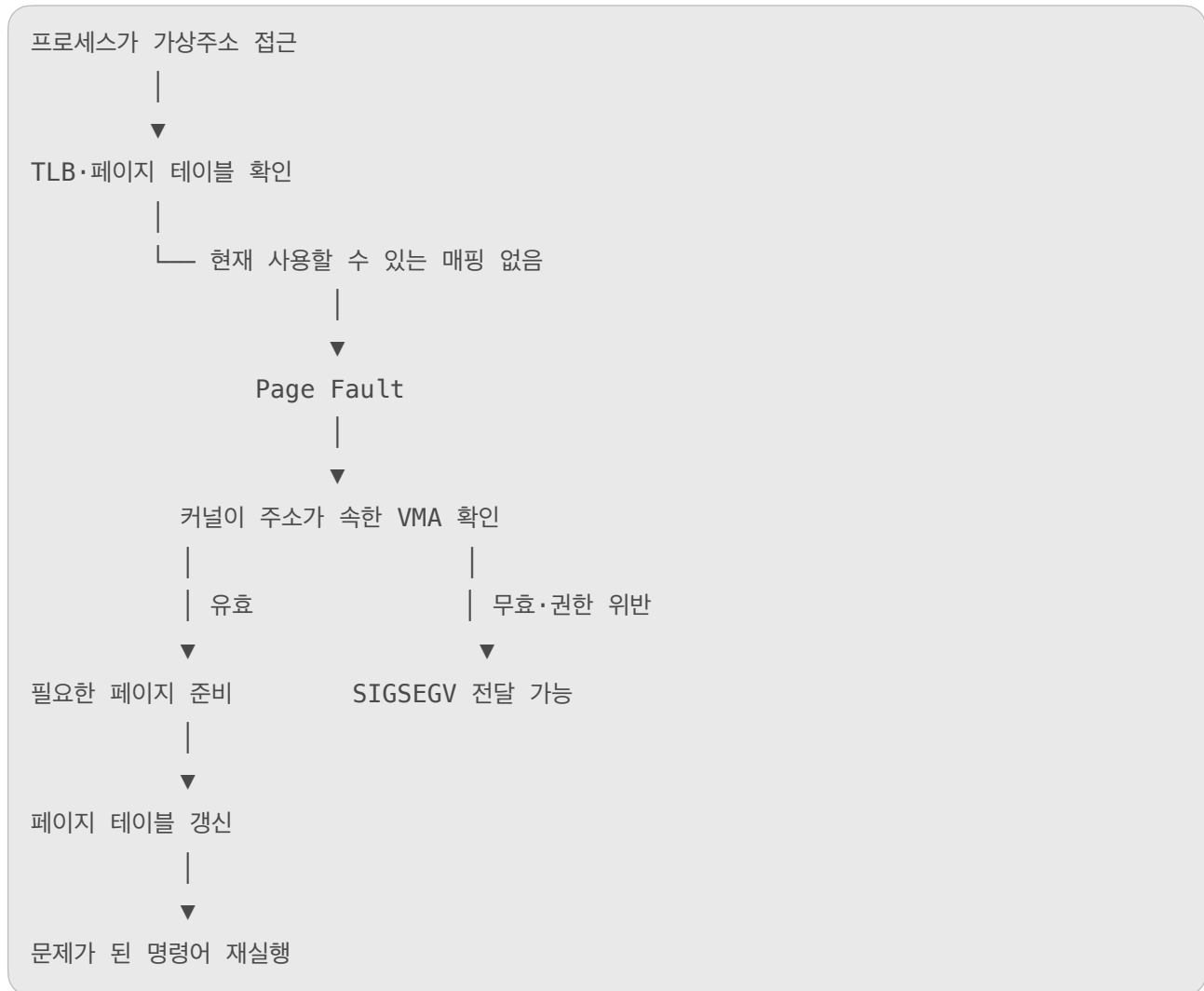
CPU 예외는 같은 상태에서 같은 명령어를 다시 실행하면 같은 지점에서 발생하므로 동기적이다.

[11] CPU 예외의 종류

CPU 아키텍처에서는 예외를 대략 다음과 같이 구분할 수 있다.

종류	보고 시점·복귀	예
Fault	문제가 된 명령어 이전 상태로 복귀 가능	Page Fault, 보호 위반
Trap	명령어 실행 후 보고	Breakpoint
Abort	정확한 재실행이 어렵고 심각함	일부 심각한 하드웨어 오류

[Page Fault]



따라서 Page Fault 자체는 프로그램 오류가 아니다.

정상적인 Page Fault

→ 요구 페이징, Copy-on-Write, 파일 매핑 등

복구 불가능한 Page Fault

→ 잘못된 주소, 권한 위반

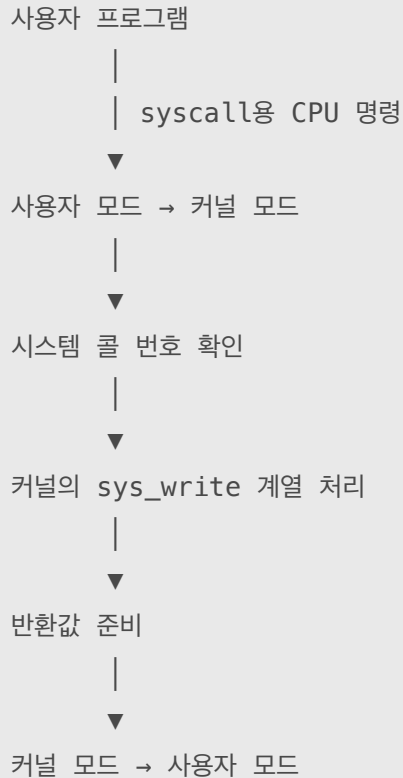
→ SIGSEGV 가능

[12] 시스템 콜

시스템 콜은 사용자 프로그램이 커널 기능을 요청하기 위해 의도적으로 커널에 진입하는 동기적 경로다.

```
write(fd, buffer, size);
```

개념적인 실행 흐름:



현대 x86-64 Linux에서는 일반적으로 다음 명령을 사용한다.

```
syscall
```

int 0x80은 x86의 전통적인 게이트 기반 시스템 콜 경로다.

현재 x86-64 프로그램의 일반적인 빠른 경로는 syscall이므로 다음처럼 정리해야 정확하다.

```
int 0x80
→ 전통적인 소프트웨어 인터럽트 명령
→ 레거시·호환 경로에서 볼 수 있음

syscall
→ 현대 x86-64의 일반적인 시스템 콜 진입 명령
→ 전통적인 하드웨어 IRQ나 Linux SoftIRQ가 아님
```

따라서 시스템 콜을 단순히 “소프트웨어 인터럽트”라고만 외우면 Linux SoftIRQ와 혼동할 수 있다.

[13] '소프트웨어 인터럽트'라는 말이 헷갈리는 이유

소프트웨어 인터럽트라는 표현은 여러 의미로 사용돼 혼란을 만든다.

소프트웨어 인터럽트 명령

→ `int 0x80`과 같이 프로그램이 실행하는 CPU 명령

동기적 CPU 예외

→ Page Fault, Breakpoint 등

Linux SoftIRQ

→ 커널의 인터럽트 후속 처리 메커니즘

이들을 다음처럼 분리해서 부르는 것이 좋다.

외부 장치 사건 → 하드웨어 IRQ

명령 실행 중 문제 → CPU 예외

커널 기능 요청 → 시스템 콜

커널 지연 처리 → Linux SoftIRQ

[14] IRQ Affinity(요청 친밀도)

IRQ Affinity는 "인터럽트 요청 선호도"보다는 다음 표현이 더 정확하다.

특정 하드웨어 IRQ를 처리할 수 있는 CPU들의 대상 마스크

NIC Queue 0 IRQ → CPU 0

NIC Queue 1 IRQ → CPU 1

NVMe Queue 0 IRQ → CPU 2

USB IRQ → CPU 3

특정 IRQ를 어느 CPU가 처리했는지 확인한다.

```
cat /proc/interrupts
```

예시:

	CPU0	CPU1	CPU2	CPU3	
24:	500000	0	0	0	eth0-TxRx-0
25:	0	320000	0	0	eth0-TxRx-1
26:	0	0	140000	0	nvme0q1

의미:

```
IRQ 24 → 주로 CPU 0에서 처리
IRQ 25 → 주로 CPU 1에서 처리
IRQ 26 → 주로 CPU 2에서 처리
```

Affinity 확인:

```
cat /proc/irq/24/smp_affinity_list
```

예:

```
0
```

IRQ 24의 처리 대상으로 CPU 0이 설정된 것이다.

설정 예시:

```
echo 0 | sudo tee /proc/irq/24/smp_affinity_list
```

여러 CPU 지정:

```
echo 0-3 | sudo tee /proc/irq/24/smp_affinity_list
```

하드웨어나 IRQ 칩의 제약 때문에 요청한 affinity와 실제 적용된 affinity가 다를 수 있다.
Generic IRQ 계층은 이를 위해 effective affinity 개념도 제공한다.

[15] 멀티큐 네트워크 카드와 IRQ 분산

현대 NIC는 여러 개의 RX/TX 큐를 가질 수 있다.

[NIC]

```
RX Queue 0 — IRQ 40 — CPU 0
RX Queue 1 — IRQ 41 — CPU 1
RX Queue 2 — IRQ 42 — CPU 2
RX Queue 3 — IRQ 43 — CPU 3
```

RSS(Receive-Side Scaling)는 패킷 헤더를 해시하여 서로 다른 RX 큐로 분산할 수 있다.

```
TCP Flow A → RX Queue 0 → CPU 0
TCP Flow B → RX Queue 1 → CPU 1
TCP Flow C → RX Queue 2 → CPU 2
TCP Flow D → RX Queue 3 → CPU 3
```

이렇게 하면 하나의 CPU에 모든 패킷 처리가 몰리는 것을 줄일 수 있다.
Linux는 RSS 외에도 RPS, RFS, XPS 등의 네트워크 CPU 분산 방식을 제공한다.

[16] 왜 인터럽트 전용 코어를 항상 사용하지 않을까?

[이유 1] 인터럽트가 적으면 코어가 낭비된다.

CPU 0: IRQ 전용, 대부분 Idle
CPU 1: 사용자 프로그램 100%
CPU 2: 사용자 프로그램 100%
CPU 3: 사용자 프로그램 100%

인터럽트가 적다면 CPU 0도 일반 작업을 실행하도록 하는 것이 전체 처리량에 유리할 수 있다.

[이유 2] 한 CPU가 병목이 될 수 있다.

CPU 0: IRQ + SoftIRQ 100%
CPU 1: 30%
CPU 2: 25%
CPU 3: 20%

모든 네트워크 IRQ를 CPU 0에 몰아두면 CPU 0이 패킷을 처리하지 못해 지연과 드롭이 발생할 수 있다.

[이유 3] 캐시 지역성이 달라진다.

CPU 0
└─ NIC IRQ와 패킷 처리
 |
 └─ CPU 0 캐시에 데이터

CPU 1
└─ 애플리케이션이 데이터 사용
 |
 └─ CPU 간 캐시 이동 발생 가능

반대로 IRQ 처리와 애플리케이션을 적절히 가까운 CPU에 배치하면 캐시 효율이 좋아질 수 있다.
다만 같은 CPU에 모든 일을 몰면 처리 경쟁이 생기므로 지역성과 부하 분산을 함께 고려해야 한다.

[17] 모든 사건을 IRQ 전용 CPU에 보낼 수는 없다.

IRQ Affinity로 분산할 수 있는 대상은 기본적으로 외부 하드웨어 IRQ다.

외부 하드웨어 IRQ

→ 인터럽트 컨트롤러와 affinity를 통해 특정 CPU로 라우팅 가능

그러나 CPU 예외와 시스템 콜은 해당 명령어를 실행하던 CPU에서 처리해야 한다.

CPU 2에서 Page Fault 발생



CPU 2가 커널의 Page Fault Handler 실행

CPU 3에서 syscall 실행



CPU 3에서 시스템 콜 처리 시작

사건	처리 CPU
외부 장치 IRQ	Affinity에 따라 대상 CPU로 전달 가능
Page Fault	명령어를 실행하던 CPU
0으로 나누기	명령어를 실행하던 CPU
시스템 콜	호출 스레드가 실행 중이던 CPU
NMI·Machine Check	아키텍처 규칙에 따라 특별 처리

[18] irqbalance

일반적인 Linux 시스템에서는 irqbalance가 하드웨어 IRQ 부하를 CPU에 분산할 수 있다.

확인:

```
systemctl status irqbalance
```

하지만 모든 IRQ를 무조건 균등하게 나누는 것은 아니다.

다음 요소를 고려한 정책과 장치 제약이 적용될 수 있다.

- CPU 토폴로지
- NUMA 노드
- 캐시 구조
- 장치 위치
- IRQ 부하
- 관리자가 지정한 affinity
- 드라이버가 관리하는 managed IRQ

따라서 irqbalance가 실행 중이어도 /proc/interrupts 를 통해 실제 분포를 확인해야 한다.

[19] 인터럽트 전용 코어가 유용한 경우

특정 CPU를 IRQ나 커널 관리 작업에 사용하고, 지연 시간에 민감한 애플리케이션을 다른 CPU에 고정하는 전략은 실제로 사용된다.

CPU 0

└─ 운영체제 housekeeping

└─ 일부 하드웨어 IRQ

CPU 1

└─ 네트워크 IRQ·SoftIRQ

CPU 2~3

└─ 지연 시간에 민감한 애플리케이션

사용 환경:

- 실시간 시스템
- 고속 패킷 처리
- 통신 장비
- 초저지연 금융 시스템
- 고성능 스토리지
- 지연 편차를 줄여야 하는 서버

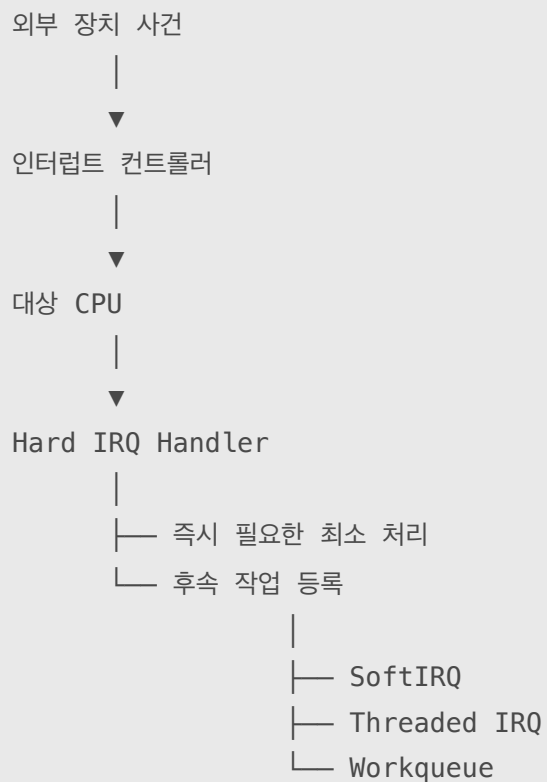
애플리케이션 고정:

```
taskset -c 2,3 ./server
```

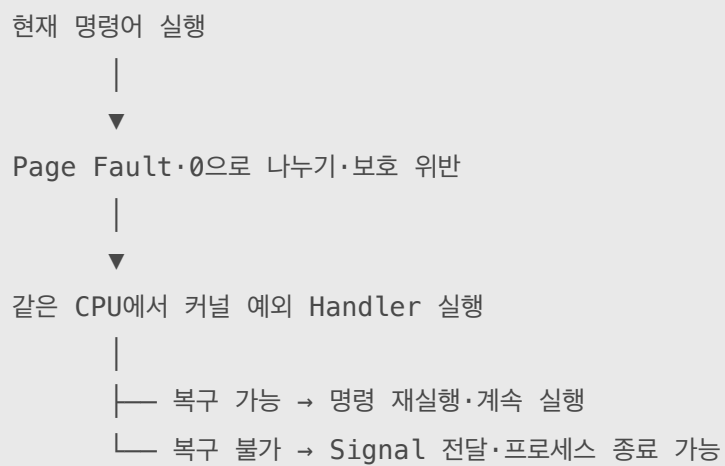
하지만 taskset만 설정한다고 완전한 CPU 격리가 되는 것은 아니다.
IRQ, 커널 스레드, Workqueue, RCU, 타이머 등도 함께 고려해야 한다.

[20] 전체 정리

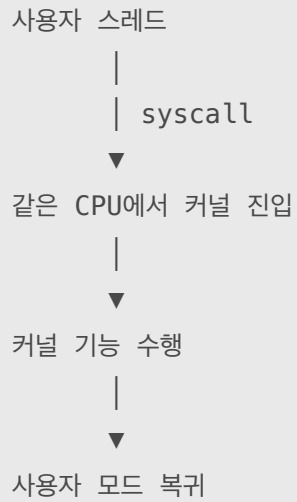
[하드웨어 IRQ]



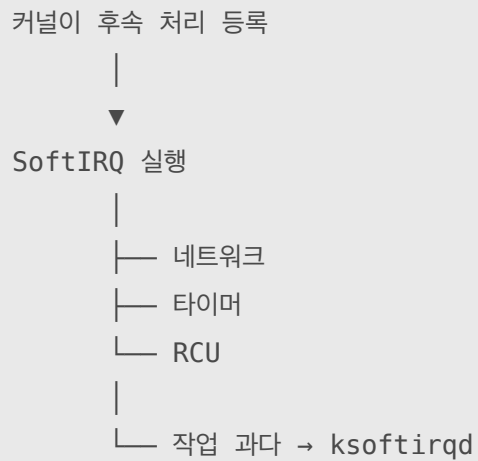
[CPU 예외]



[시스템 콜]

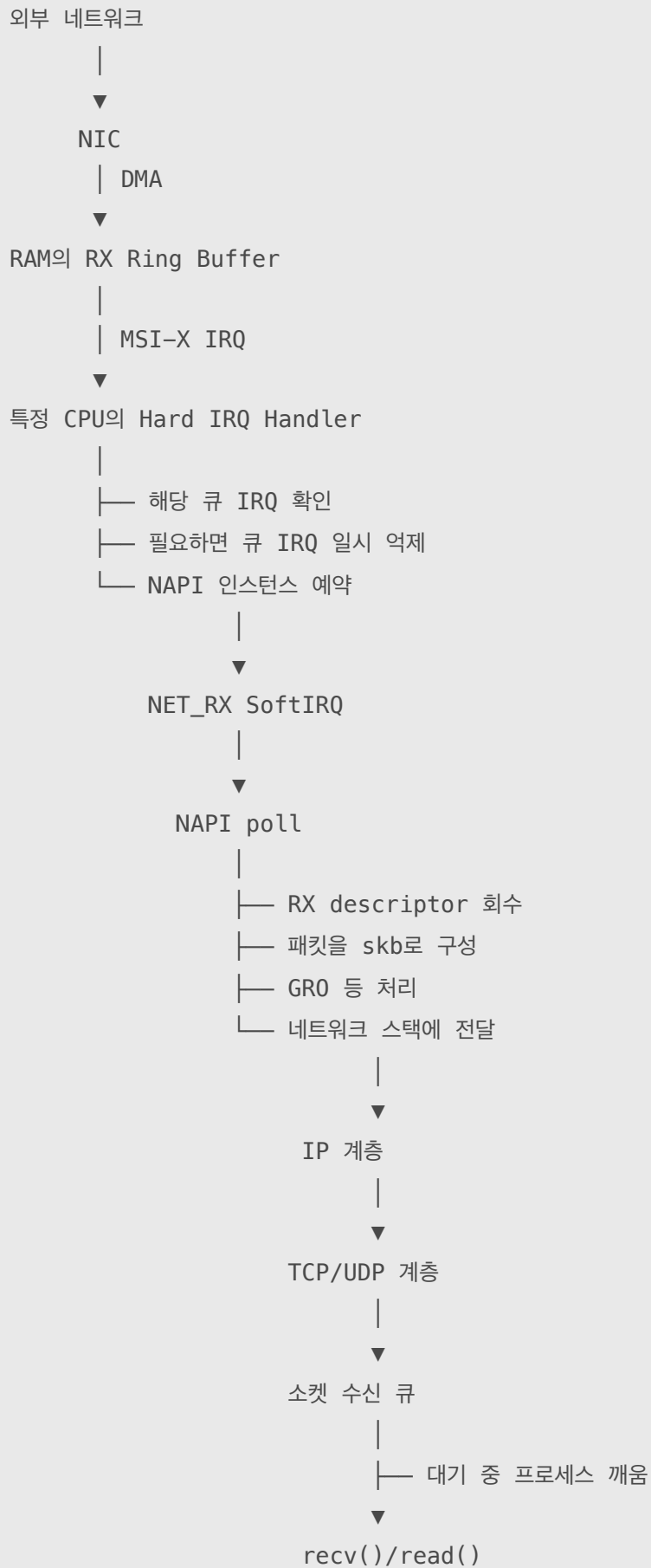


[Linux SoftIRQ]



[21] 네트워크 패킷 처리의 실제 흐름

[패킷 수신]



↓
사용자 버퍼

DMA(Direct Memory Access)

- CPU를 거치지 않고 하드웨어 장치(디스크, 네트워크 카드 등)가 시스템 메모리(RAM)에 직접 데이터를 읽고 쓸 수 있게 해주는 기술이다. CPU가 데이터 이동에 낭비되는 시간을 없애 시스템 성능을 높여준다.

RX 링 버퍼(Receive Ring Buffer)

- 네트워크 카드(NIC)가 케이블을 통해 들어오는 패킷을 커널이 처리하기 전 1차로 저장하는 임시 고정 크기 순환 메모리 공간이다.

MSI-X(Message Signaled Interrupts-Extended)

- PCIe 장치가 최대 2048개의 개별 독립적 IRQ(인터럽트) 벡터를 할당받아 고속 네트워크 카드(NIC) 등의 멀티 큐 성능을 극대화하는 최신 인터럽트 방식이다.
- 기존 공유 방식과 달리 각 큐나 CPU 코어에 인터럽트를 전용으로 할당할 수 있다.

PCIe(PCI Express)

- 그래픽 카드, NVMe SSD, 네트워크 카드 같은 고속 하드웨어 장치들을 메인보드 및 CPU와 연결해 주는 고속 직렬 인터페이스 규격이자, 이를 관리하는 리눅스 커널의 PCI/Pcie 서브시스템을 의미한다.

NAPI의 기본 흐름은 Hard IRQ -> SoftIRQ -> NAPI Poll이다.

장치는 먼저 IRQ로 사건을 알리고, 커널은 NAPI를 예약하여 일정량의 패킷을 폴링 방식으로 처리한다.

[왜 NAPI를 사용하는가?]

패킷마다 IRQ가 하나씩 발생한다면 고속 트래픽에서 문제가 생긴다.

패킷 1 → IRQ
패킷 2 → IRQ
패킷 3 → IRQ
패킷 4 → IRQ
...

트래픽이 많을 때 NAPI는 IRQ를 억제하고 한 번에 여러 패킷을 처리한다.

한 번의 IRQ
↓
NAPI Poll
├─ 패킷 1
├─ 패킷 2
├─ 패킷 3
└─ 패킷 N

이를 통해 인터럽트 폭주를 줄이고 처리량을 높인다.

NAPI Poll이 정해진 budget 안에서 모든 패킷을 처리하면 완료하고 IRQ를 다시 활성화한다.

작업이 남으면 계속 스케줄된 상태로 다음 처리를 이어간다.